

Arhitectura calculatoarelor

Curs

Cristian Vancea

E-mail: Cristian.Vancea@cs.utcluj.ro

Web: <http://users.utcluj.ro/~vcristian/AC.html>

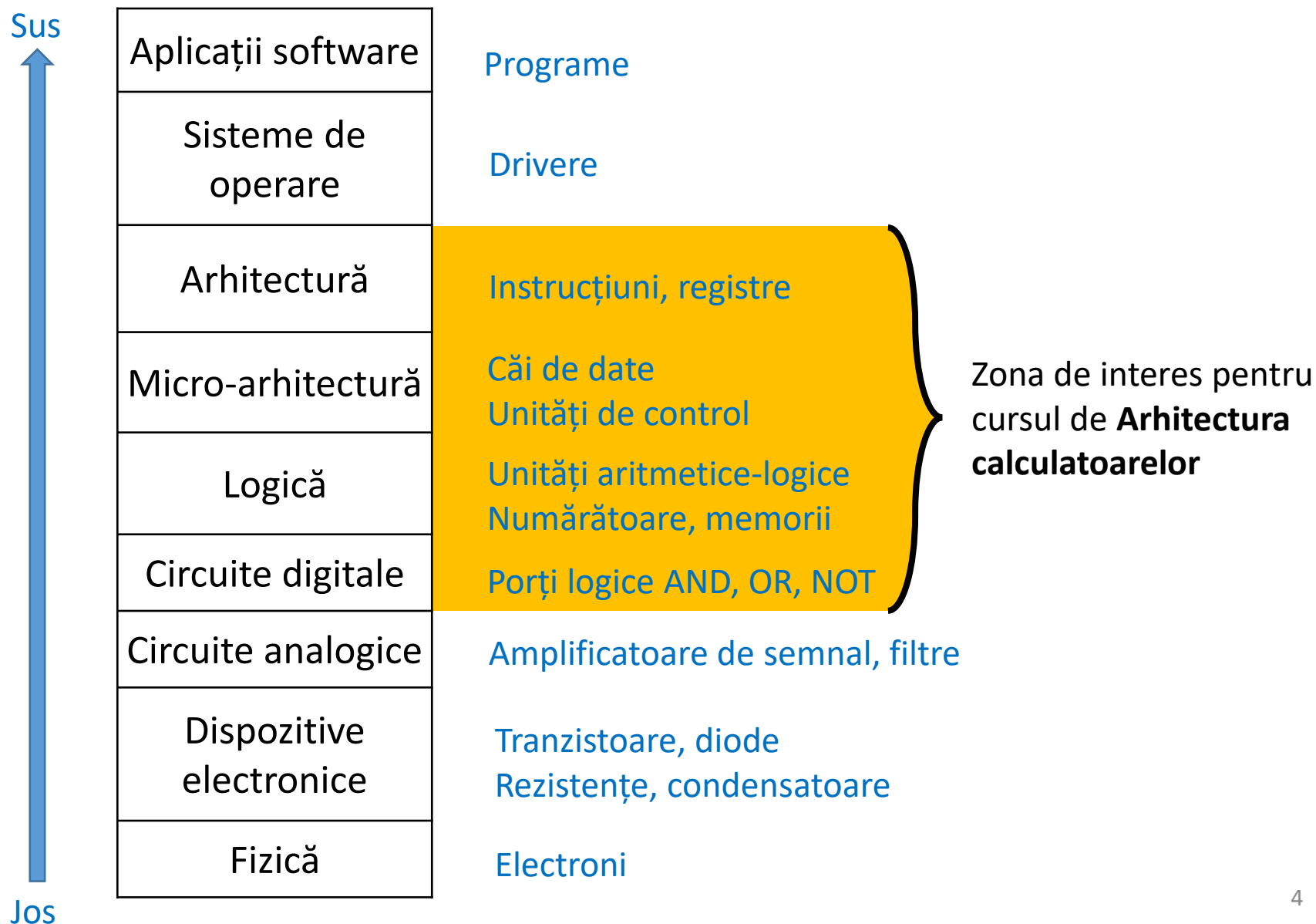
Obiectivele generale ale cursului

- Studiul metodelor de proiectare a sistemelor digitale.
- Specificarea Arhitecturii Setului de Instrucțiuni.
- Dezvoltarea de programe în limbaj de asamblare și cod mașină.
- Descrierea, implementarea și testarea Unității Centrale de Prelucrare (UCP).
- Studiul organizării memoriei.
- Analiza tendințelor moderne în arhitectura calculatoarelor.

Conținutul cursului

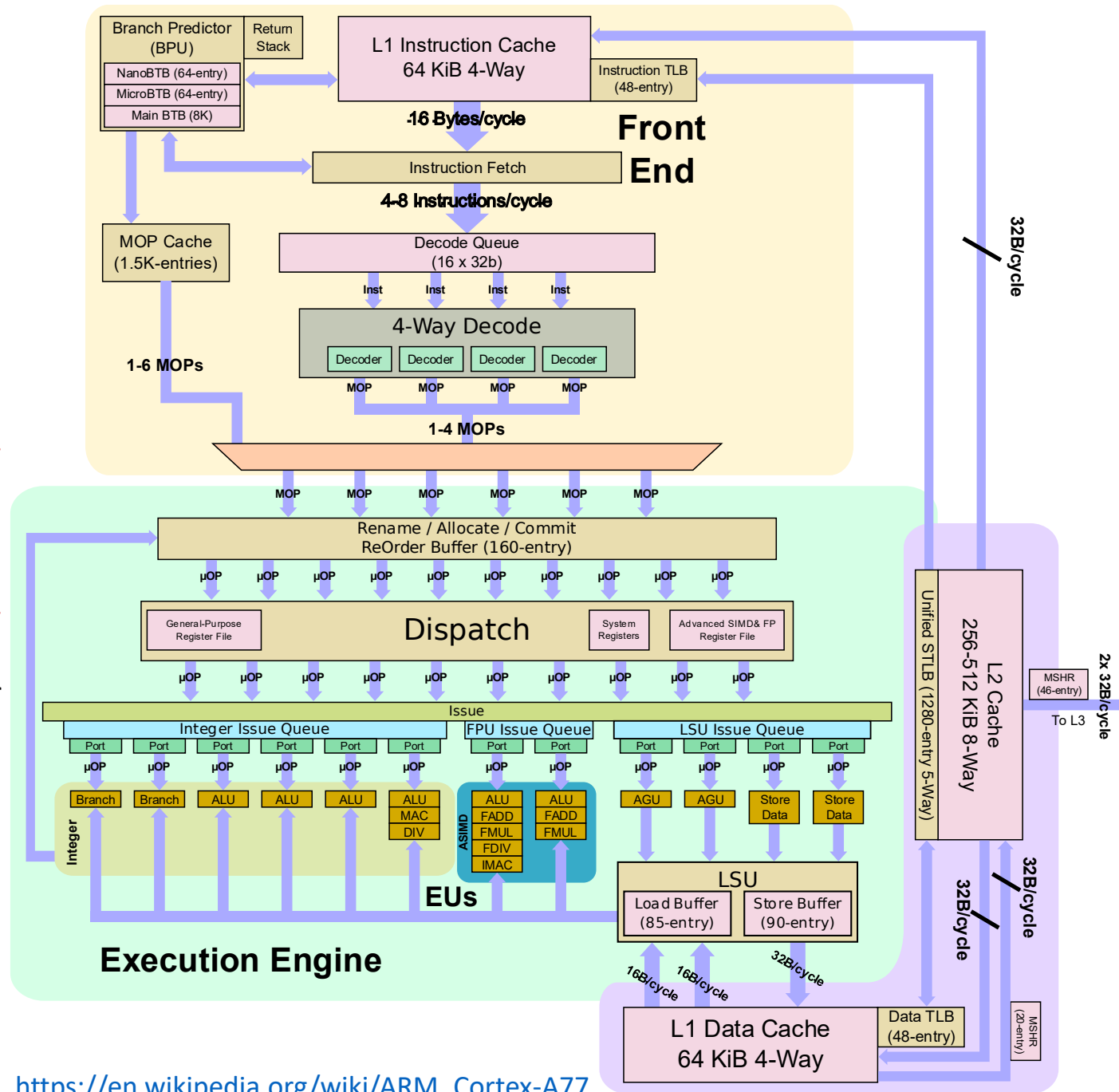
1. Introducere
2. Proiectarea sistemelor digitale
3. Arhitectura Setului de Instrucțiuni
4. Proiectarea procesorului cu un singur ciclu de execuție
5. Aritmetica calculatoarelor, unități aritmetice și logice
6. Procesorul cu cicluri multiple de execuție – calea de date
7. Procesorul cu cicluri multiple de execuție – controlul
8. Procesorul pipeline. Hazarduri
9. Planificarea dinamică a execuției
10. Execuție speculativă și predicția ramificărilor
11. Memorii
12. Arhitecturi moderne de procesoare
13. Rezolvare de probleme

Niveluri de abstractizare în sistemele de calcul



Exemplu:
Micro-arhitectura
ARM Cortex-A77

- **ISA:** ARMv8.2.
- Conține 1-8 nuclee cu **13 niveluri pipeline**.
- Tehnologie: 5, 7, 10 nm.
- Execuție **dinamică**.
- **Cache:**
 - Lvl1 Date – 64KB (**4-way**);
 - Lvl1 Instrucțiuni – 64KB (**4-way**);
 - Lvl2 – 256-512KB (**8-way**);
 - Lvl3 – 0-4MB/cluster (**16-way**).



Obiectivele lucrărilor de laborator

- Operarea cu conceptele învățate la curs.
- Deprinderea de abilități practice pentru:
 - proiectare de arhitecturi de microprocesor și descriere în VHDL;
 - scriere de programe în asamblare și cod-mașină;
 - testare/simulare a funcționării pe plăci didactice și în mediul de implementare.

Notă₁: Lucrările de laborator sunt componente obligatorii ale disciplinei Arhitectura calculatoarelor.

Notă₂: Lucrările de laborator se studiază în prealabil !!!

Notă₃: Conținutul se găsește pe site!

Conținutul laboratorului

- Utilizarea mediului de dezvoltare Vivado
- Circuite logice combinaționale
- Circuite logice secvențiale
- Procesorul MIPS cu ciclu unic
- Procesorul MIPS pipeline
- Comunicare serială

Bibliografie

- [1] F. Oniga, “De la bit la procesor. Introducere în arhitectura calculatoarelor”, Ed. U.T. Press, 2019, ISBN 978-606-737-366-0.
- [2] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 3rd edition (2004), 5th edition (2013), RISC-V edition (2017), Ed. Morgan–Kaufmann.
- [3] D. A. Patterson, J. L. Hennessy, “Organizarea și proiectarea calculatoarelor. Interfața Hardware/Software”, Ed. ALL, 2002, ISBN: 973-684-444-7.
- [4] J. L. Hennessy, D. A. Patterson, “Computer Architecture: A Quantitative Approach”, 5th edition (2011), 6th edition (2017), Ed. Morgan–Kaufmann.
- [5] S. L. Harris, D. M. Harris, “Digital Design and Computer Architecture”, ARM edition (2015), RISC-V edition (2021), Ed. Morgan–Kaufmann.
- [6] W. Stallings, “Computer Organization and Architecture”, 11th edition, global edition, Ed. Pearson, 2021.
- [7] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.
- [8] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.

Evaluare

- Examen (E)
 - onsite: scris
 - online: scris și/sau oral
- Laborator (L)
 - onsite: scris și/sau oral
 - online: scris și/sau oral

Nota = $\text{round}((E + L)/2)$, $\text{round}()$ – rotunjire la cel mai apropiat întreg

Condiția de promovare: $L \geq 5p$ și $E \geq 5$

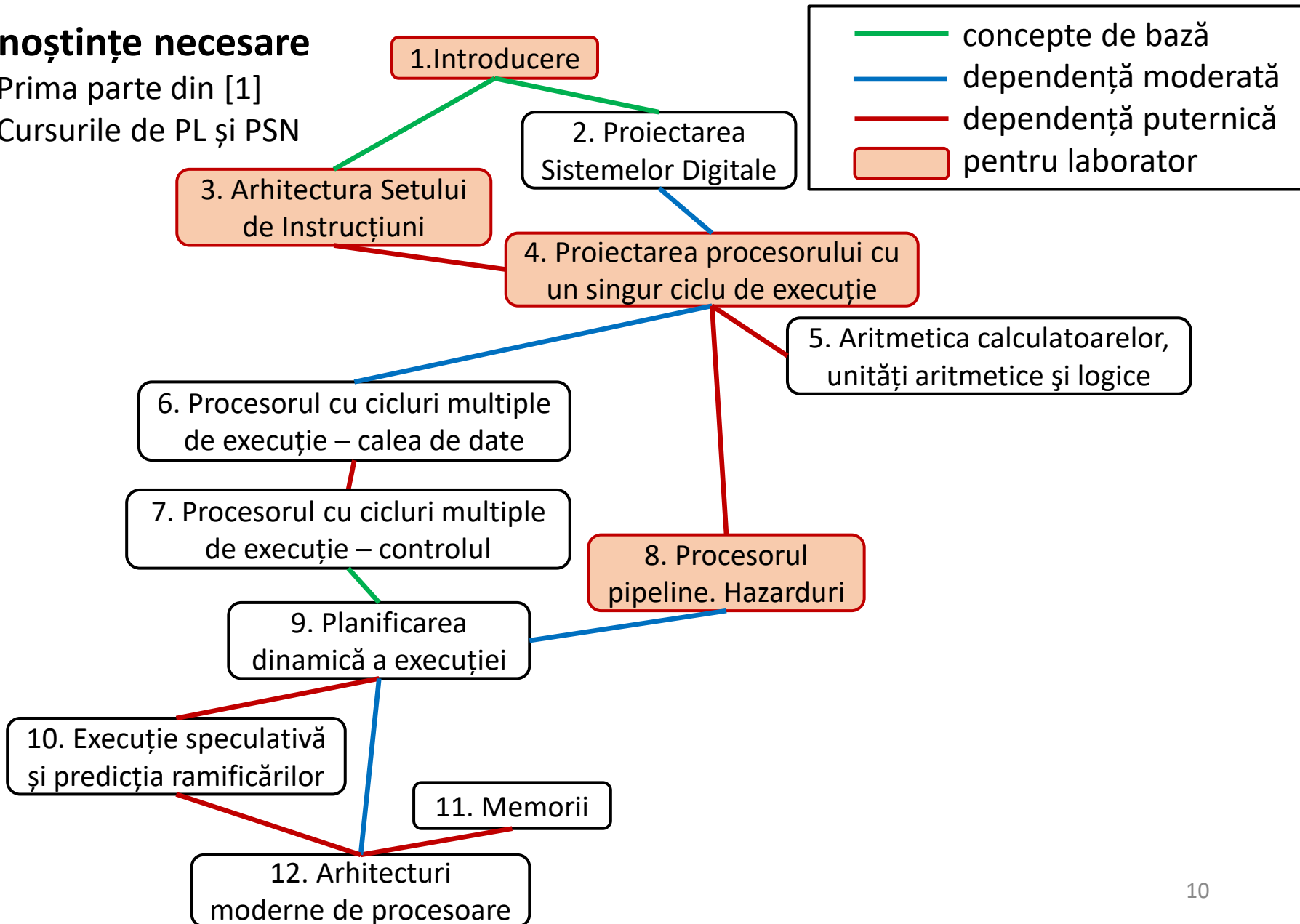
Structura examenului scris:

1. Test întrebări (grilă sau cu calcule) din toate cursurile.
2. Probleme rezolvate pe larg (eseu) – sigur o problemă din MIPS ciclu unic.

Dependența cursurilor

Cunoștințe necesare

- Prima parte din [1]
- Cursurile de PL și PSN



Arhitectura calculatoarelor

Curs 1

1. Introducere

Cristian Vancea

E-mail: Cristian.Vancea@cs.utcluj.ro

Web: <http://users.utcluj.ro/~vcristian/AC.html>

Obiective

1. Concepte de bază
2. Paralelism și arhitecturi paralele
3. Arhitectura unui procesor
4. Arhitectura Setului de Instrucțiuni (ISA)
5. Evaluarea performanței
6. Descrierea în VHDL (recapitulare)

1.1 Concepte de bază

- Arhitectura Calculatoarelor urmărește:
 - Arhitectura Setului de Instrucțiuni (Instruction Set Architecture - ISA);
 - Organizarea calculatorului (micro-arhitectura).
- ISA:
 - interfața între hardware și software, la nivelul de jos.
- Micro-arhitectura:
 - componente și conexiunile dintre ele;
 - Registre, unități ALU, memorii, etc.
- Aceeași ISA poate avea micro-arhitecturi diferite:
 - Diferă prin preț, performanță, energie consumată;
 - Ex: MIPS ciclu unic, MIPS multi ciclu, MIPS pipeline.
- Exemple ISA:
 - IA-32, IA-64, PowerPC, MIPS, SPARC, ARM.

➡ Microprocessor without Interlocked Pipelined Stages

➡ Advanced RISC Machines 13

1.2 Paralelism și arhitecturi paralele

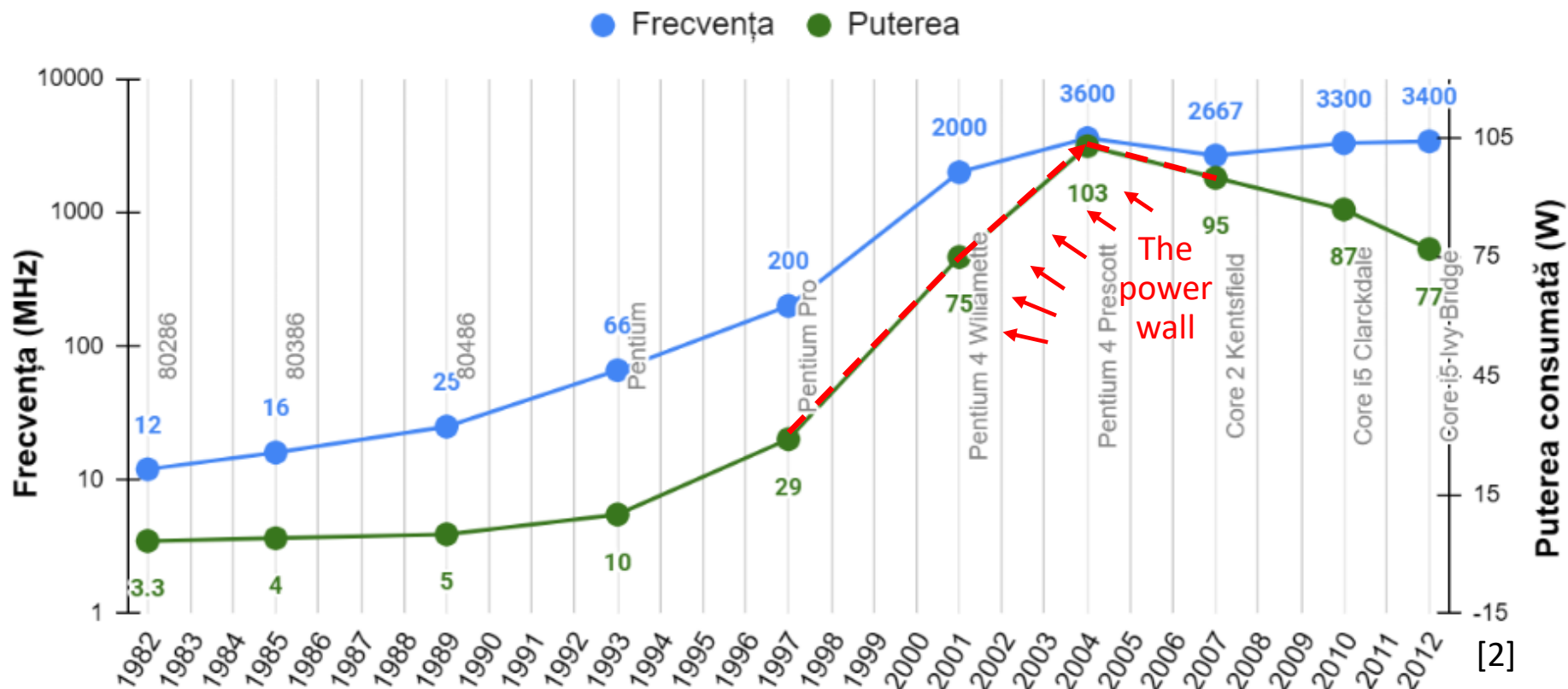
Motivație

→ putere consumată

→ tranzistori pe ieșire (fanout)

→ clock rate

- Definiție: $\text{Power/transistor} = \frac{1}{2} \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$



[2]

- Anul 2004 – Intel Pentium 4 înregistrează o creștere puternică a consumului de putere datorită ridicării frecvenței, dar creșterea performanței este sub așteptări.
- Intel **renunță la creșterea frecvenței** în favoarea înmulțirii nucleelor de procesare => tehnologia urmează o stagnare a frecvenței cu îmbunătățirea consumului.

1.2 Parallelism și arhitecturi paralele

- Tipuri de paralelism în programe:
 - la nivel de date – Data Level Parallelism (DLP);
 - la nivel de task-uri – Task Level Parallelism (TLP).
- Exploatarea paralelismului:
 1. **Instruction Level Parallelism (ILP)**
 - paralelism de date;
 - **pipelining, execuție speculativă.**
 2. **Vector Architectures & Graphic Processor Units (GPU)**
 - paralelism de date;
 - se aplică o instrucțiune pe seturi de date, în paralel.
 3. **Thread-Level Parallelism**
 - paralelism de date și task;
 - programe cu thread-uri paralele care interschimbă date.
 4. **Request-Level Parallelism**
 - paralelism între task-uri decuplate: thread-uri ale sistemului de operare comunică cu thread-uri ale programelor utilizator.

1.2 Paralelism și arhitecturi paralele

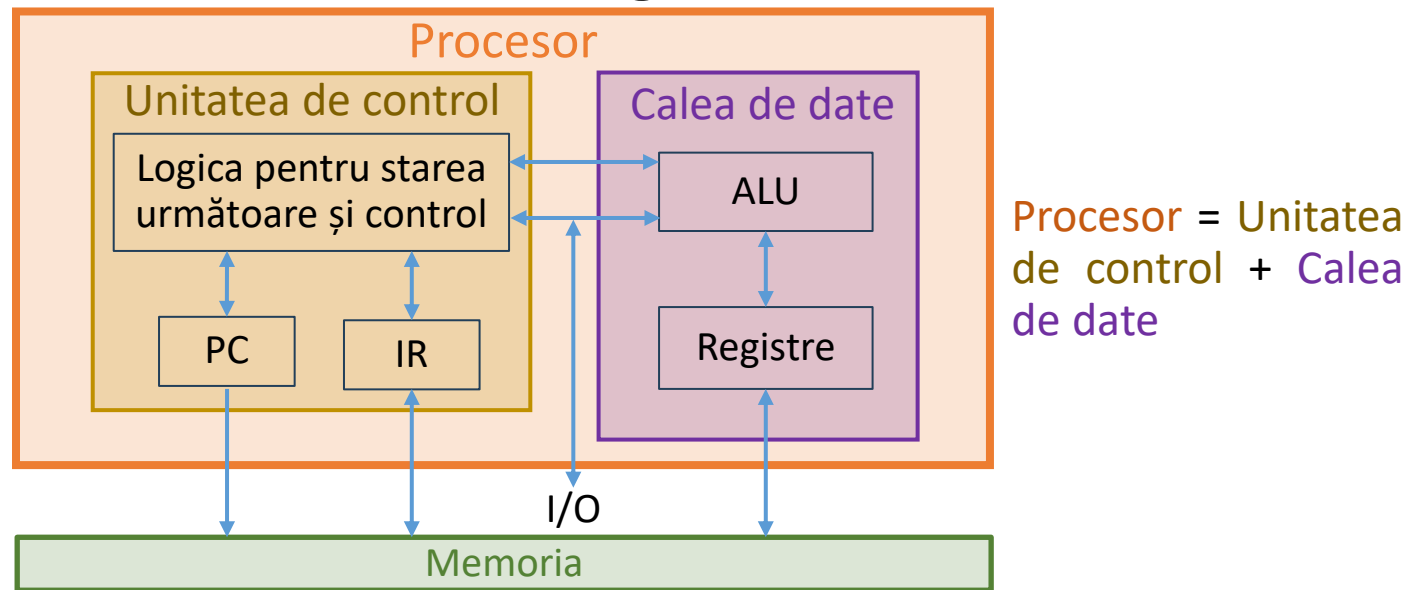
- Taxonomia lui Flynn

Taxonomia lui Flynn		
	Single Instruction	Multiple Instruction
Single Data	SISD	MISD
Multiple Data	SIMD	MIMD

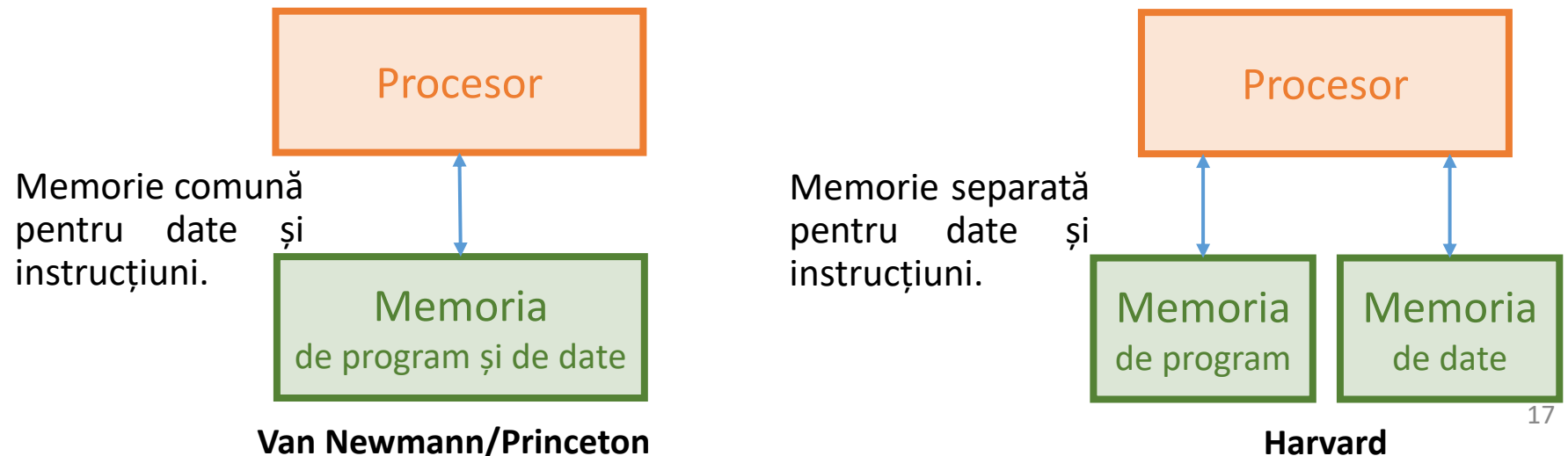
- SISD – uniprocessor, execuție secvențială, poate implementa **pipelining**, **execuție speculativă** (ILP);
- SIMD – o instrucțiune executată pe mai multe unități de procesare cu date diferite (arhitecturi vectoriale + GPU);
- MISD – redundanță de calcul folosită pentru siguranță la defecțiuni;
- MIMD – procesoare cu instrucțiuni și date proprii, exploatează TLP.

1.3 Arhitectura unui procesor

1.3.1 Arhitectura generală



1.3.2 Arhitecturi clasice



1.3 Arhitectura unui procesor

1.3.3 Tipuri de arhitecturi

1. Complex Instruction Set Computer (CISC)

- Instrucțiuni multe și complexe;
- Registre puține;
- ALU cu operanzi în registre și memorie (registru-memorie);
- Multe moduri de adresare a memoriei;
- Necesită arhitectură complexă pentru pipeline (paralelizare).

2. Reduced Instruction Set Computer (RISC)

- Instrucțiuni mai puține ;
- Registre multe;
- ALU cu operanzi în registre (registru-registru);
- Puține moduri de adresare a memoriei cu instrucțiuni load/store;
- Necesită arhitectură simplă pentru pipeline (paralelizare).

1.4 Arhitectura Setului de Instrucțiuni (ISA)

Definiție: Interfața între hardware și software, la nivelul de jos.

Componente

- Organizarea memoriei și registrele disponibile;
- Adresarea memoriei: pentru memoria de date și cea de instrucțiuni;
- Tipuri de date: codificări, reprezentări;
- Setul de instrucțiuni și formatul acestora;
- Tratarea excepțiilor, interacțiunea cu perifericele.

Majoritatea ISA moderne sunt **General Purpose Register** (GPR) – operațiile ALU au loc între registre sau locații de memorie. Clase de ISA GPR:

- register-memory ISA: operații ALU registru-registru și registru-memorie
 - Ex: x86 (cunoscută și ca 80x86)
- **load-store ISA**: operații ALU registru-registru, acces la memorie cu load/store
 - Ex: **MIPS**, ARM

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.1 Adresarea memoriei

	Mod de adresare	Exemplu	Efect
L-S	Register	ADD R4, R3	$R4 \leftarrow R4 + R3$
	Immediate	ADD R4, 3	$R4 \leftarrow R4 + 3$
	Displacement	ADD R4, 100(R1)	$R4 \leftarrow R4 + \text{Mem}[100 + R1]$
	Register indirect	ADD R4, (R1)	$R4 \leftarrow R4 + \text{Mem}[R1]$
R-M	Indexed & Base	ADD R3, (R1+R2)	$R3 \leftarrow R3 + \text{Mem}[R1 + R2]$
	Direct or Absolute	ADD R1, (100)	$R1 \leftarrow R1 + \text{Mem}[100]$
	Memory Indirect	ADD R1, @(R3)	$R1 \leftarrow R1 + \text{Mem}[\text{Mem}[R3]]$
	Auto-increment	ADD R1, (R2)+	$R1 \leftarrow R1 + \text{Mem}[R2]$ $R2 \leftarrow R2 + d$
	Auto-decrement	ADD R1, (R2)-	$R2 \leftarrow R2 - d$ $R1 \leftarrow R1 + \text{Mem}[R2]$

d – dimensiunea unui cuvânt de memorie în octeți

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.2 Tipuri de date (x86, MIPS, ARM)

- 8-bit – char;
- 16-bit – short int;
- 32-bit – int, float (virgulă mobilă cu simplă precizie);
- 64-bit – long int, double (virgulă mobilă cu dublă precizie).

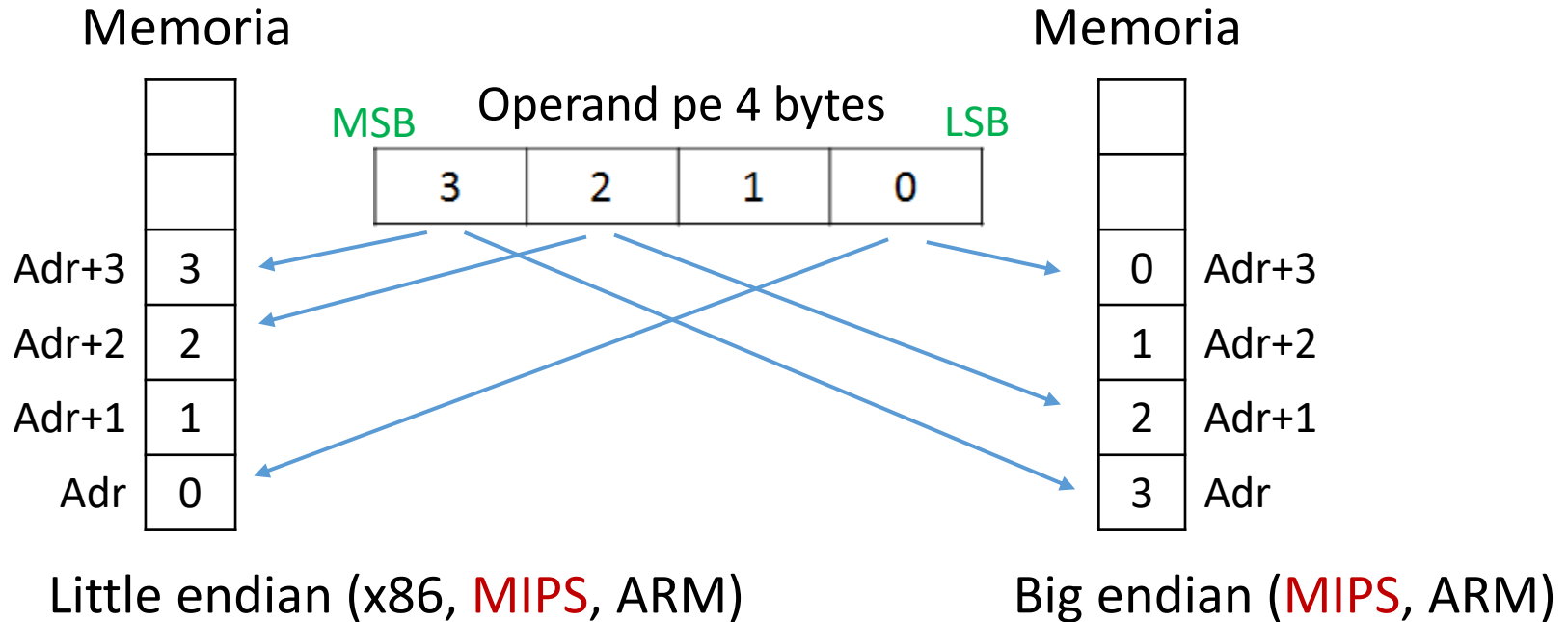
1.4.3 Alinierea în memorie

Dacă se accesează un obiect de N octeți, adresa trebuie să fie multiplu de N .

- x86 – nu impune aliniere, dar accesul este mai rapid cu aliniere;
- MIPS, ARM – **necesită aliniere**.

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.4 Stocarea în memorie a operanzilor



Care byte se pune la adresa mică?

- LSB => little endian
- MSB => big endian

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.5 Setul de instrucțiuni

Tipuri de operații	Descriere / Exemple
Aritmetice și logice cu întregi	ADD, SUB, AND, OR, MUL, DIV
Transfer de date	Diferite forme de load (mem -> reg), store (reg -> mem), move (mem -> mem, reg -> reg)
Control	Salt condiționat (branch), salt necondiționat (jump), apeluri de rutine (call) și revenire din rutine (return)
Sistem	Inițiate de sistemul de operare sau legate de management-ul memoriei virtuale
Floating Point	Cu numere fracționare în virgulă mobilă: adunare, înmulțire, împărțire, comparații
Decimal	Operații în baza 10: adunare, înmulțire, conversie la caracter
String	Operații pe șiruri de caractere: move, compare, search
Graphics	Operații pe pixeli, vectori, compresie, decompresie

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.5 Setul de instrucțiuni

Instrucțiuni de control

- Salturi condiționate – **branch**:
 - Salt la un număr de instrucțiuni față de cea curentă (adresare relativă);
 - Pentru validarea saltului se testează:
 - biții de stare (flags) – la x86;
 - **conținutul registrelor** – la **MIPS**.
- Salturi necondiționate – **jump**:
 - Salt la o adresă în program (adresare absolută);
- Apel de rutine și revenire – **call**, **return**. Adresa de revenire se plasează în:
 - memorie (stivă) – la x86;
 - **registru** – la **MIPS** și ARM.

Formatul instrucțiunilor

- x86: lungime variabilă pe 1-18 octeți, implică decodificare complexă și programul ocupă spațiu redus în memorie;
- **MIPS**, ARM: **lungime fixă pe 4 octeți** (32 de biți), implică decodificare simplă și programul ocupă spațiu mai mare în memorie.

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.5 Setul de instrucțiuni (cont)

Formatul instrucțiunilor

- Variabil – x86, VAX

Operation and no. of operands	Address specifier 1	Address field 1		Address specifier n	Address field n
-------------------------------	---------------------	-----------------	-------	-----------------------	-------------------

- Fix – MIPS, ARM

Operation	Address field 1	Address field 2	Address field 3
-----------	-----------------	-----------------	-----------------

- Hibrid – MIPS16, Thumb

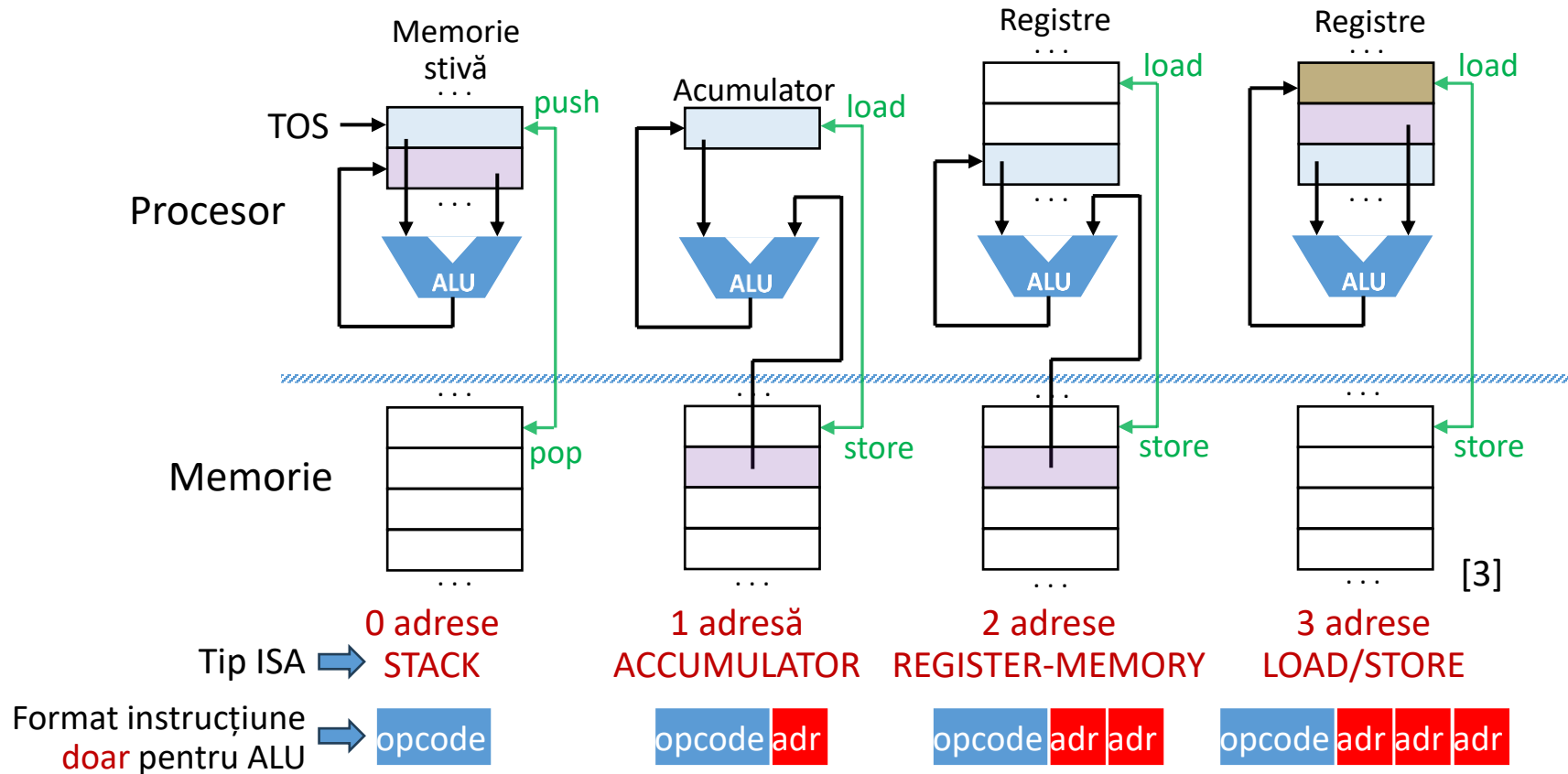
Operation	Address specifier	Address field	
-----------	-------------------	---------------	--

Operation	Address specifier 1	Address specifier 2	Address field
-----------	---------------------	---------------------	---------------

Operation	Address specifier	Address field 1	Address field 2
-----------	-------------------	-----------------	-----------------

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.6 Localizarea operanzilor



1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.6 Localizarea operanzilor (cont)

Stack

- Registrul TOS (Top Of Stack) indică vârful stivei. Operațiile ALU au loc întotdeauna între vârful stivei și locația imediat sub ea; TOS se auto-decrementează și rezultatul se scrie în stivă la noua locație TOS.
- Push/Pop realizează auto-incrementare/decrementare TOS și apoi transfer între memorie și Stack[TOS]. **Notă: Toți operanzii sunt implicați.**

Accumulator

- Operațiile ALU au loc între acumulator și un operand din memorie; rezultatul se scrie în acumulator. Load/Store realizează transfer între memorie și acumulator. **Notă: Acumulatorul este operand implicit.**

Register-Memory

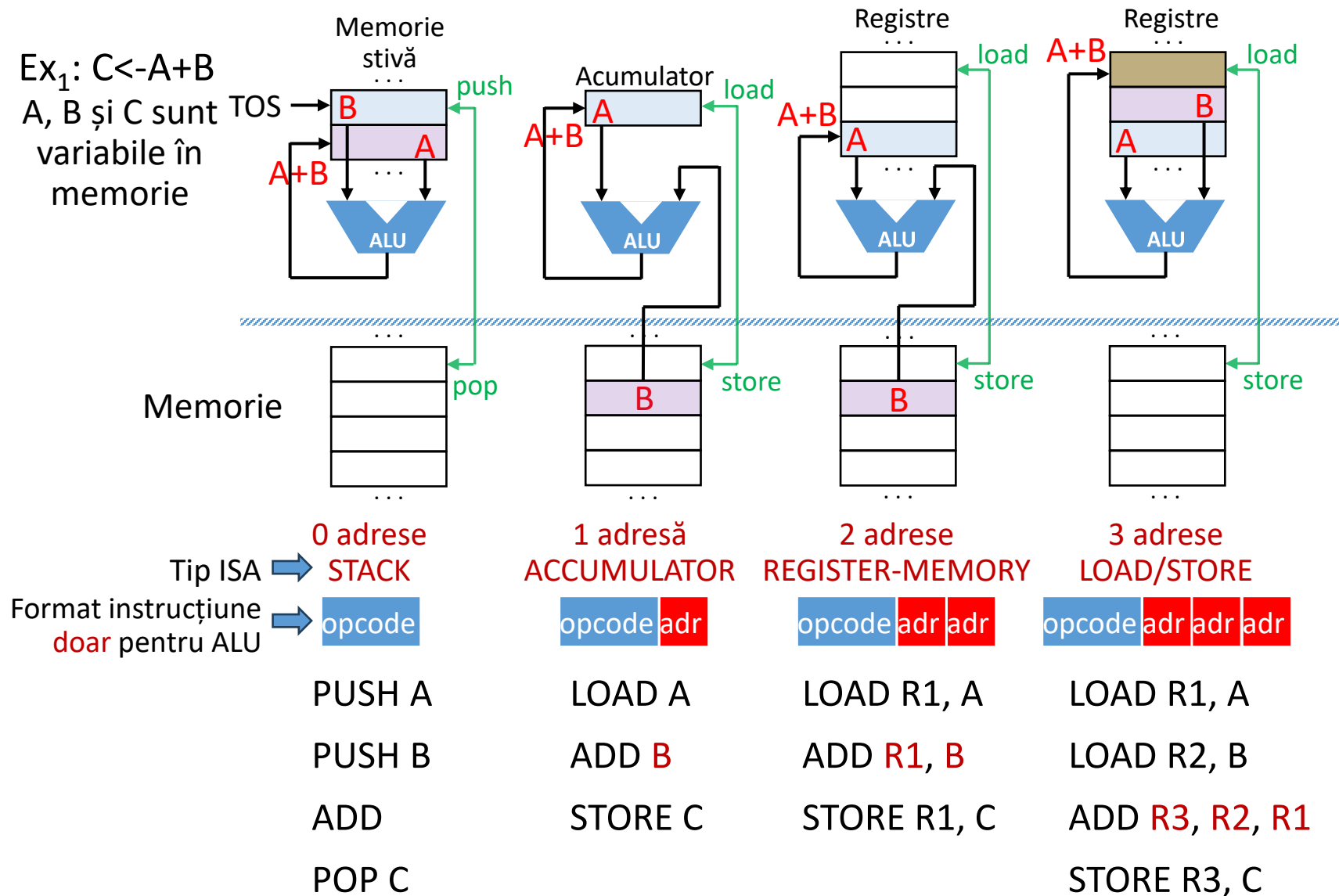
- Operațiile ALU au loc între un registru și un operand din memorie; rezultatul se scrie înapoi în registru. Load/Store => transfer între memorie și registre.

Load/Store

- Operațiile ALU au loc între 2 registre, cu scrierea rezultatului în oricare din registre. Load/Store => transfer între memorie și registre.

1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.6 Localizarea operanzilor (cont)

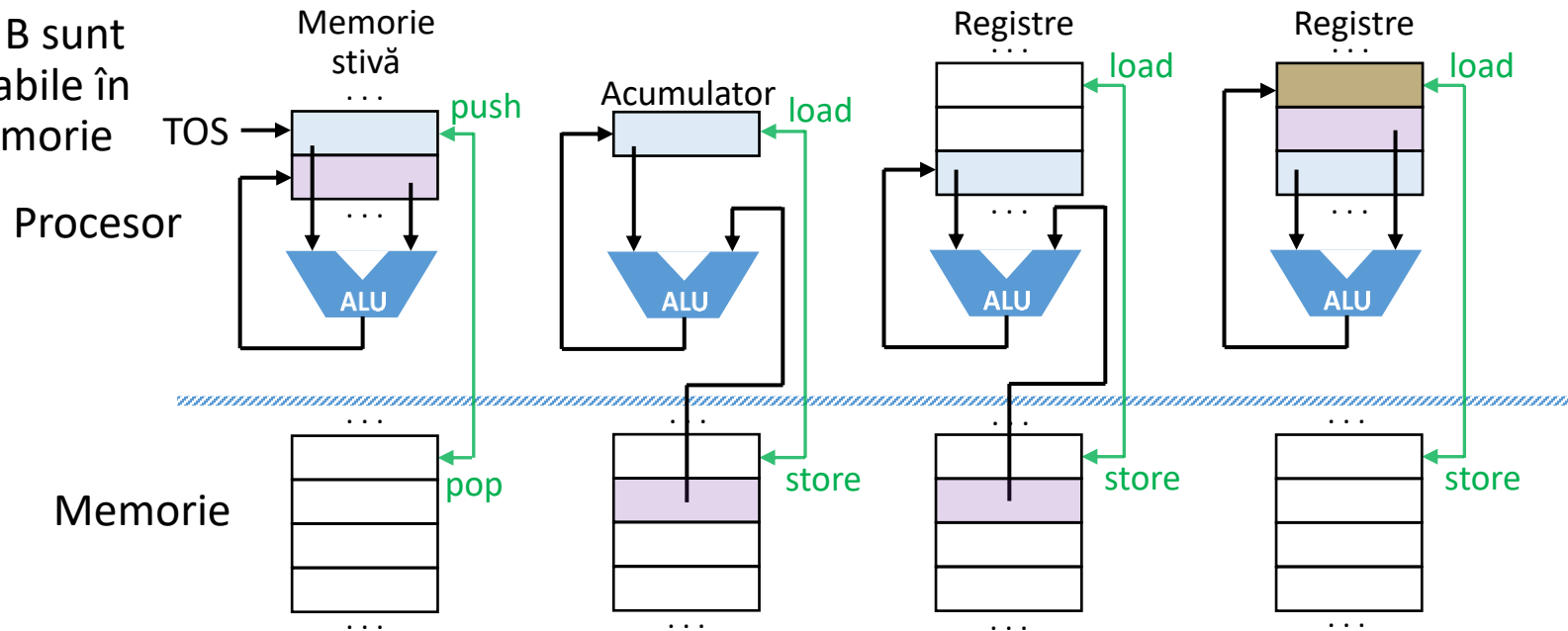


1.4 Arhitectura Setului de Instrucțiuni (ISA)

1.4.6 Localizarea operanzilor (cont)

Ex₂: $A \leftarrow -A + B + B$

A și B sunt
variabile în
memorie



	0 adrese	1 adresă	2 adrese	3 adrese
Tip ISA	→ STACK	ACCUMULATOR	REGISTER-MEMORY	LOAD/STORE
Format instrucțiune doar pentru ALU	→ opcode	opcode adr	opcode adr adr	opcode adr adr adr
PUSH A		LOAD A	LOAD R1, A	LOAD R1, A
PUSH B		ADD B	ADD R1, B	LOAD R2, B
PUSH B		ADD B	ADD R1, B	ADD R1, R1, R2
ADD		STORE A	STORE R1, A	ADD R1, R1, R2
ADD				STORE R1, A
POP A				

1.5 Evaluarea performanței

- **Bandwidth (lățime de bandă) sau Throughput (productivitate)**

- Măsoară numărul de rezultate într-un interval de timp;
- Ex: transferul de date pe disk se măsoară în MB/s.

- **Latency (întârziere) sau Response time (timp de răspuns)**

- Timpul necesar efectuării unui eveniment;
- Ex: accesul la disk se măsoară în milisecunde.

- **Timpul CPU pentru un program**

- Durata execuției programului:

$$CPU_{time} = IC \cdot CPI \cdot CCT = \frac{Instructions}{Program} \cdot \frac{Cycles}{Instruction} \cdot \frac{Seconds}{Cycles} = \frac{Seconds}{Program}$$

IC (Instruction Count) – numărul de instrucțiuni; influențat de ISA și compilator

CPI (Cycles Per Instruction) – cicluri de ceas / instrucțiune; influențat de ISA

CCT (Clock Cycle Time) – perioada de ceas; influențat de hardware

- **Performanța : $Perf = 1 / CPU_{time}$**

1.5 Evaluarea performanței

- **Viteza de execuție**

- Se măsoară în MIPS (Millions Instructions Per Second)

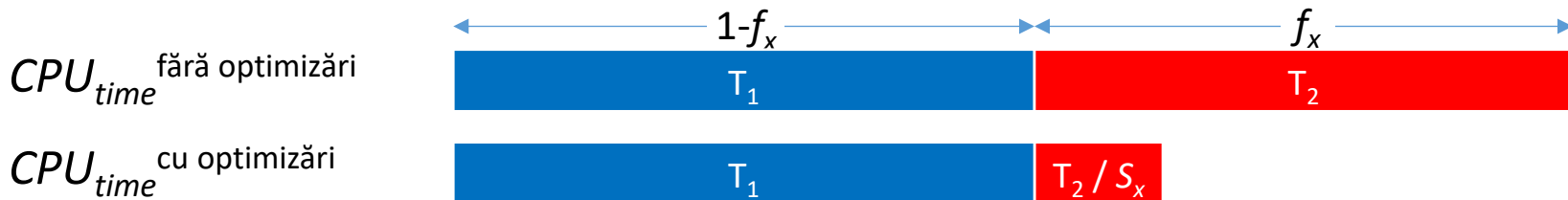
$$MIPS = \frac{IC/10^6}{CPU_{time}} = \frac{IC}{IC \cdot CPI \cdot CCT \cdot 10^6} = \frac{1}{CPI \cdot \frac{1}{Clock\ rate} \cdot 10^6} = \frac{Clock\ rate}{CPI \cdot 10^6}$$

- **Speedup (creșterea vitezei)**

$$Speedup = \frac{Perf^{cu\ optimizări}}{Perf^{fără\ optimizări}} = \frac{CPU_{time}^{fără\ optimizări}}{CPU_{time}^{cu\ optimizări}}$$

Legea lui Amdahl

Îmbunătățirea performanței (speedup) prin optimizarea unei părți este limitată de fracțiunea de timp în care partea optimizată este folosită.



Fracțiunea de timp îmbunătățită cu speedup S_x este $f_x \Rightarrow \begin{cases} T_1 = (1-f_x) \cdot CPU_{time}^{fără\ optimizări} \\ T_2 = f_x \cdot CPU_{time}^{fără\ optimizări} \end{cases}$

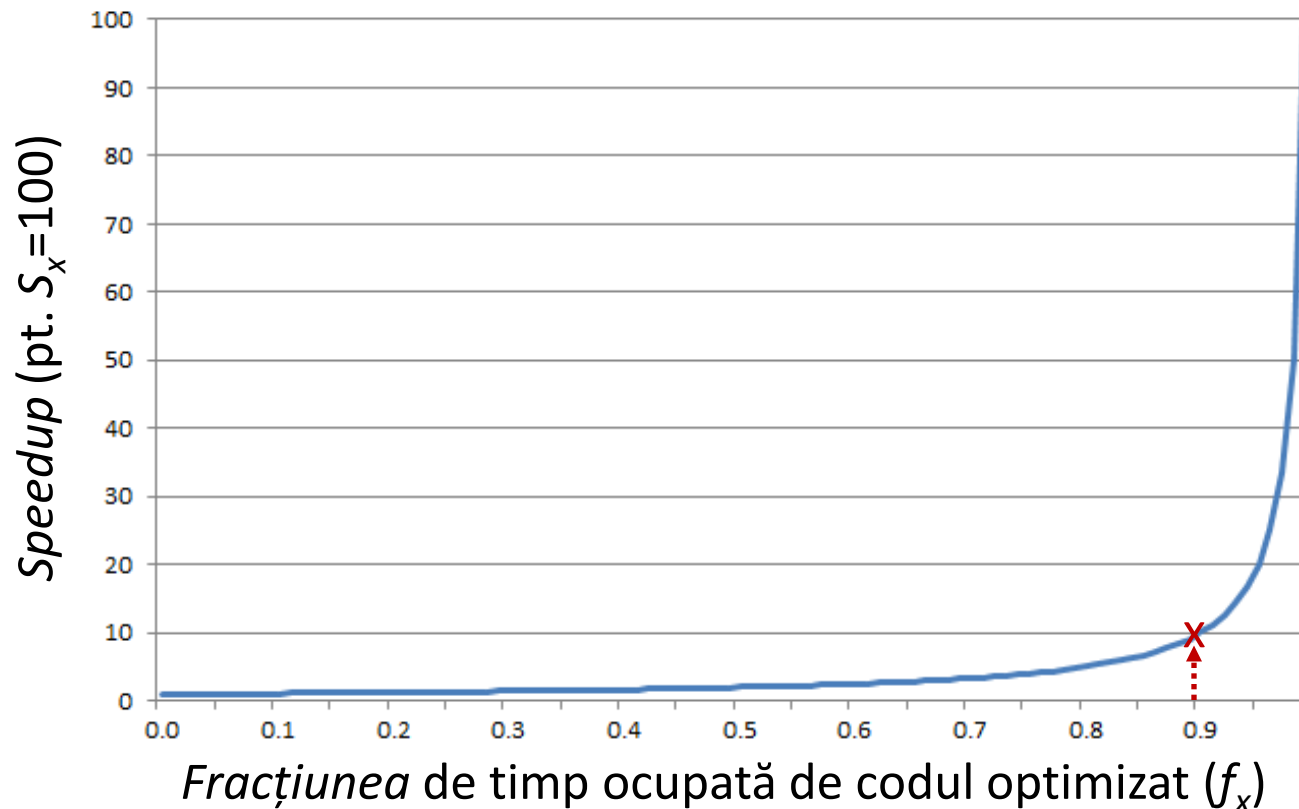
$$CPU_{time}^{cu\ optimizări} = CPU_{time}^{fără\ optimizări} \cdot \left[(1-f_x) + \frac{f_x}{S_x} \right]$$

1.5 Evaluarea performanței

- **Speedup (creșterea vitezei)** (cont)

$$CPU_{time}^{\text{cu optimizări}} = CPU_{time}^{\text{fără optimizări}} \cdot \left[(1-f_x) + \frac{f_x}{S_x} \right]$$

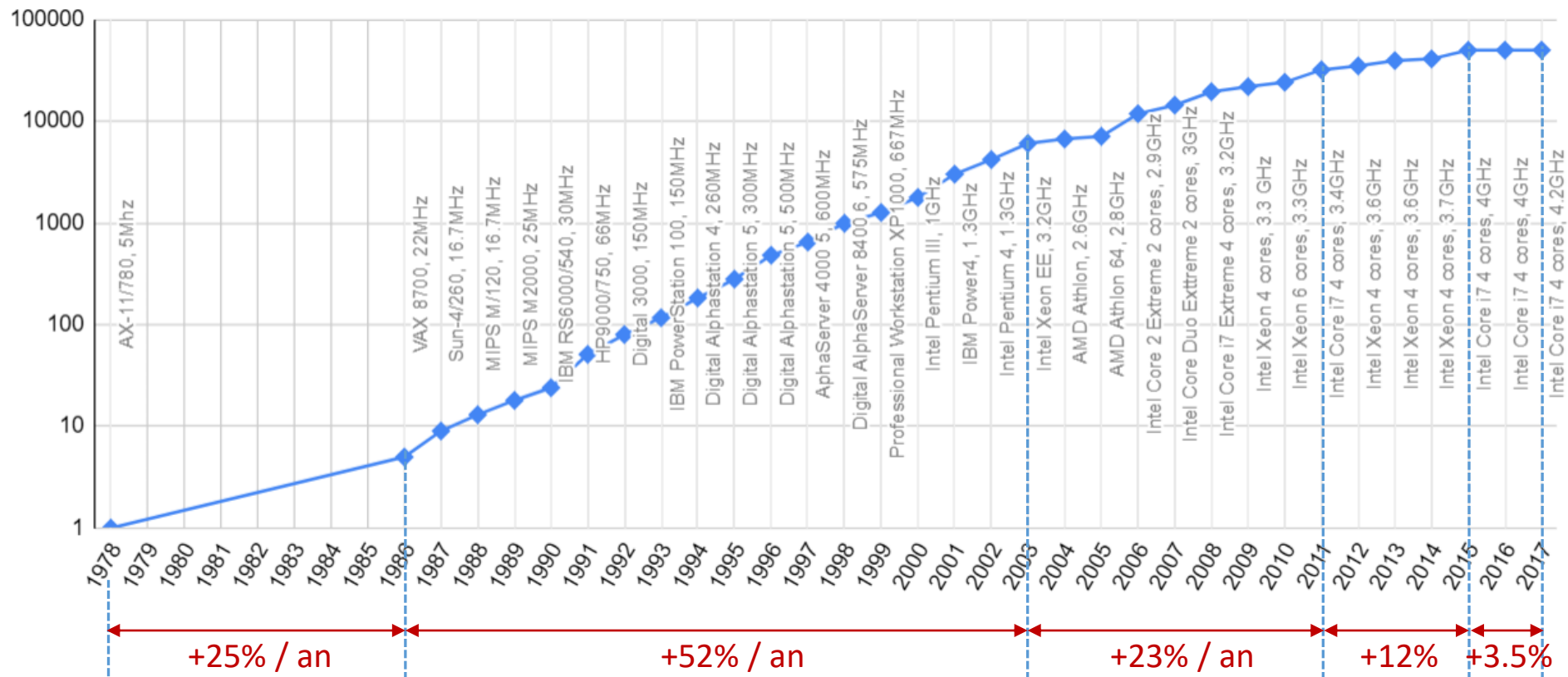
$$Speedup = \frac{CPU_{time}^{\text{fără optimizări}}}{CPU_{time}^{\text{cu optimizări}}} = \frac{CPU_{time}^{\text{fără optimizări}}}{CPU_{time}^{\text{fără optimizări}} \cdot \left[(1-f_x) + \frac{f_x}{S_x} \right]} = \frac{1}{(1-f_x) + \frac{f_x}{S_x}}$$



Concluzie: Cel mai mare efect îl are optimizarea zonelor de program costisitoare (cu impact în peste 90% din timpul de execuție). Ex: bucle cu număr mare de iterații, operații aritmetice lente, repetitive.

1.5 Evaluarea performanței

Benchmark SPEC



[2][3]

VAX:
îmbunătățirea
tehnologiei

RISC & x86: îmbunătățirea
arhitecturii și a organizării

Limitare putere/frecvență,
concentrare pe multicore

1.6 Descrierea în VHDL (recapitulare)

1.6.1 Declaraarea semnalelor

- Semnal de 1 bit:

`signal sig_name : std_logic := '0';`

- Semnal de N biți:

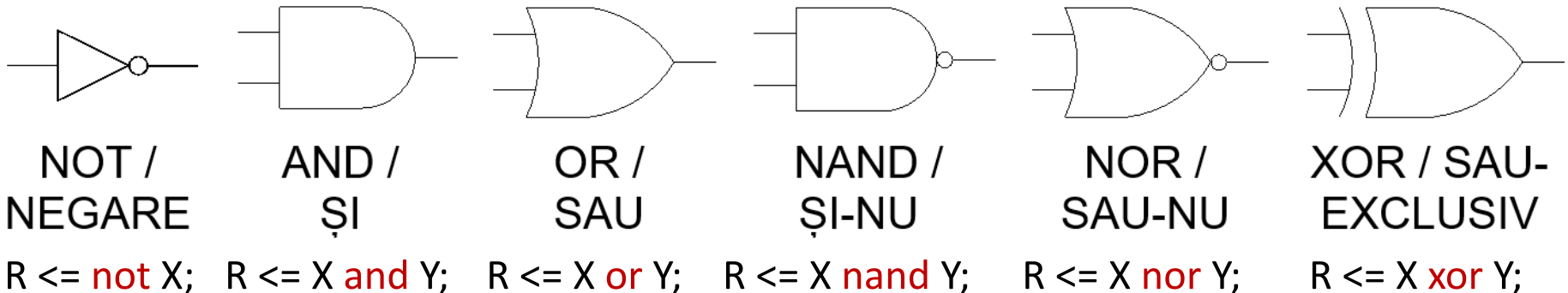
`signal sig_name : std_logic_vector(N-1 downto 0) := "00....0";`

- Exemple de inițializare:

- binară, pentru semnal pe 16 biți: `"0000000000000000"`
- hexazecimală, pentru semnal pe 32 biți: `X"00000000"`
- binară, pentru semnal generic pe N biți: `(others => '0')`

1.6.2 Circuite logice combinaționale

Porți logice fundamentale

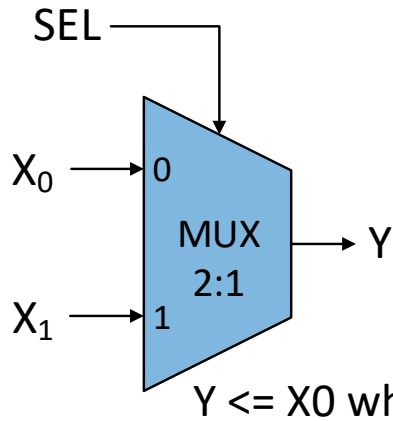


Nu folosiți entitate suplimentară, declarați semnalele în arhitectura entității principale!

1.6 Descrierea în VHDL (recapitulare)

1.6.2 Circuite logice combinaționale

Multiplexor MUX 2:1

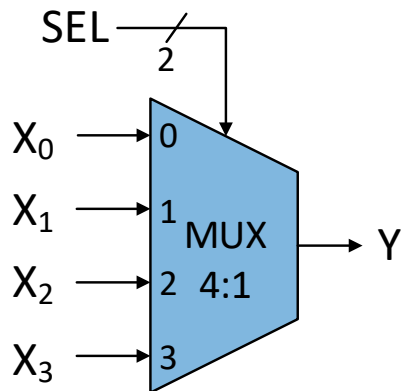


```
process(SEL, X0, X1)
begin
    if SEL = '0' then
        Y <= X0;
    else
        Y <= X1;
    end if;
end process;
```

La circuite combinaționale toate intrările se pun în lista de sensibilitate!



Multiplexor MUX 4:1



```
process(SEL, X0, X1, X2, X3)
begin
    case SEL is
        when "00" => Y <= X0;
        when "01" => Y <= X1;
        when "10" => Y <= X2;
        when "11" => Y <= X3;
    end case;
end process;
```

Nu folosiți entitate suplimentară, declarați semnalele în arhitectura entității principale!

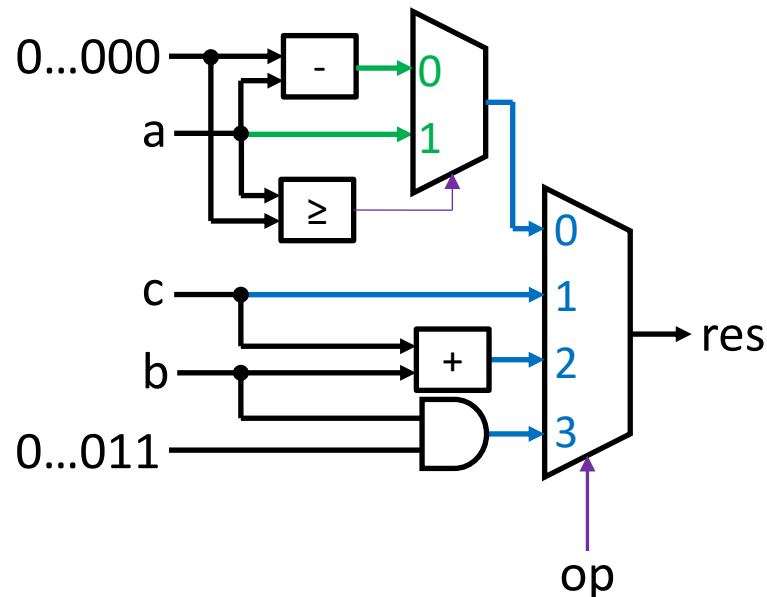
1.6 Descrierea în VHDL (recapitulare)

1.6.2 Circuite logice combinaționale (cont)

Problemă

Să se implementeze circuitul care realizează următorul pseudocod:

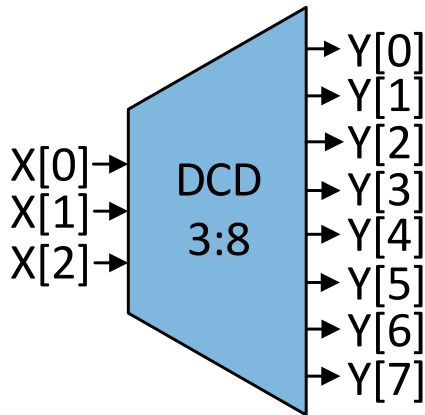
```
if op=0 then0 if a≥0 then1 res ← a  
                else0 res ← -a  
if op=1 then1 res ← c  
if op=2 then2 res ← c + b  
if op=3 then3 res ← b & 3
```



1.6 Descrierea în VHDL (recapitulare)

1.6.2 Circuite logice combinaționale (cont)

Decodificator DCD 3:8



```
process(X)
begin
  case X is
```

```
    when "000" => Y <= "00000001";
    when "001" => Y <= "00000010";
    when "010" => Y <= "00000100";
    when "011" => Y <= "00001000";
    when "100" => Y <= "00010000";
    when "101" => Y <= "00100000";
    when "110" => Y <= "01000000";
    when "111" => Y <= "10000000";
```

```
  end case;
end process;
```



La circuite combinaționale
toate intrările se pun în
lista de sensibilitate!

1.6 Descrierea în VHDL (recapitulare)

1.6.3 Circuite logice secvențiale

Registru / D Flip-Flop



```
process(clk)
begin
  if rising_edge(clk) then
    q <= d;
  end if;
end process;
```



```
process(clk)
begin
  if rising_edge(clk) then
    if en = '1' then
      q <= d;
    end if;
  end if;
end process;
```



```
process(clk)
begin
  if rising_edge(clk) then
    if rst = '1' then
      q <= "0...0";
    elsif en = '1' then
      q <= d;
    end if;
  end if;
end process;
```



```
process(clk, rsta)
begin
  if rsta = '1' then
    q <= "0...0";
  elsif rising_edge(clk) then
    if en = '1' then
      q <= d;
    end if;
  end if;
end process;
```

`rising_edge(clk) ⇔ clk'event and clk='1'`
`falling_edge(clk) ⇔ clk'event and clk='0'`



La circuite secvențiale *clk* și comenzile asincrone se pun în lista de sensibilitate.

Nu folosiți entitate suplimentară, declarați semnalele în arhitectura entității principale!

1.6 Descrierea în VHDL (recapitulare)

1.6.3 Circuite logice secvențiale (cont)

Numărător



```
process(clk)
begin
  if rising_edge(clk) then
    cnt <= cnt+1;
  end if;
end process;
```



```
process(clk)
begin
  if rising_edge(clk) then
    if en = '1' then
      cnt <= cnt+1;
    end if;
  end if;
end process;
```



```
process(clk)
begin
  if rising_edge(clk) then
    if rst = '1' then
      cnt <= "0...0";
    elsif en = '1' then
      cnt <= cnt+1;
    end if;
  end if;
end process;
```



```
process(clk, rsta)
begin
  if rsta = '1' then
    cnt <= "0...0";
  elsif rising_edge(clk) then
    if en = '1' then
      cnt <= cnt+1;
    end if;
  end if;
end process;
```

use IEEE.STD_LOGIC_UNSIGNED.ALL;

`rising_edge(clk)` $\Leftrightarrow$ `clk'event and clk='1'`
`falling_edge(clk)` $\Leftrightarrow$ `clk'event and clk='0'`



La circuite secvențiale *clk* și comenzile asincrone se pun în lista de sensibilitate.

Nu folosiți entitate suplimentară, declarați semnalele în arhitectura entității principale!

1.6 Descrierea în VHDL (recapitulare)

1.6.3 Circuite logice secvențiale (cont)

Numărător

Inițializare implicită, în lipsa unei inițializări explicite:

- Pe placă: cu valoarea "0000...0000"
- În simulatorul Vivado: cu "XXXXX...X"
 - Problemă: "XXXXX...X" + 1 = "XXXXX...X" => blocarea numărătorului pe "XXXXX...X"

Obligatoriu -> Inițializare explicită la declarare:



Ex. `signal cnt_name : STD_LOGIC_VECTOR(7 downto 0) := (others => '0');`

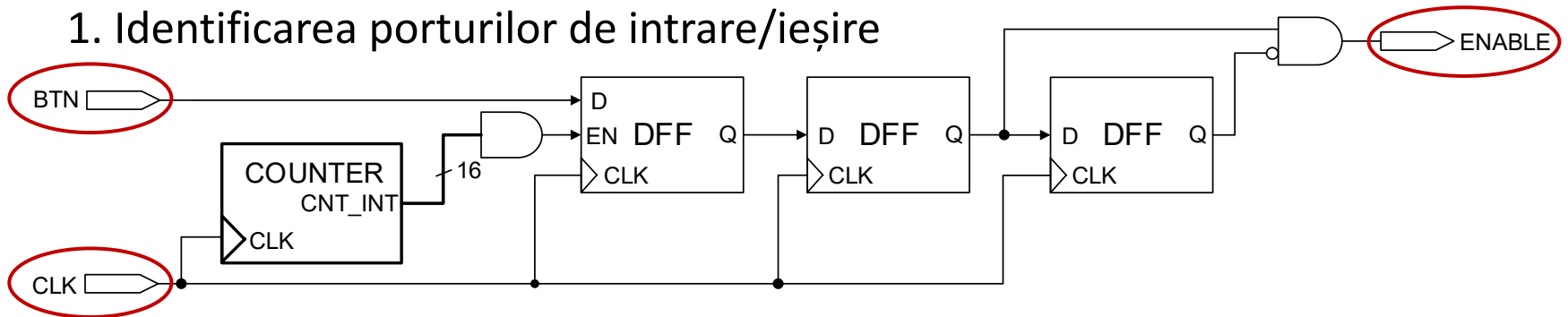
Aceeași regulă de inițializare explicită este valabilă și la **memorii**, deoarece în lipsa ei primesc "UUUUU...U" (Unknown) în simulator.

Ex. `signal mem_name : mem_type := ((others => '0'));`

1.6 Descrierea în VHDL (recapitulare)

1.6.4 De la schemă la cod VHDL – Generatorul de monoimpuls

1. Identificarea porturilor de intrare/ieșire



-- Declaraire librării

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.STD_LOGIC_UNSIGNED.ALL;
```

entity MPG is

```
Port ( enable : out STD_LOGIC;  
      btn : in STD_LOGIC;  
      clk : in STD_LOGIC);
```

end MPG;

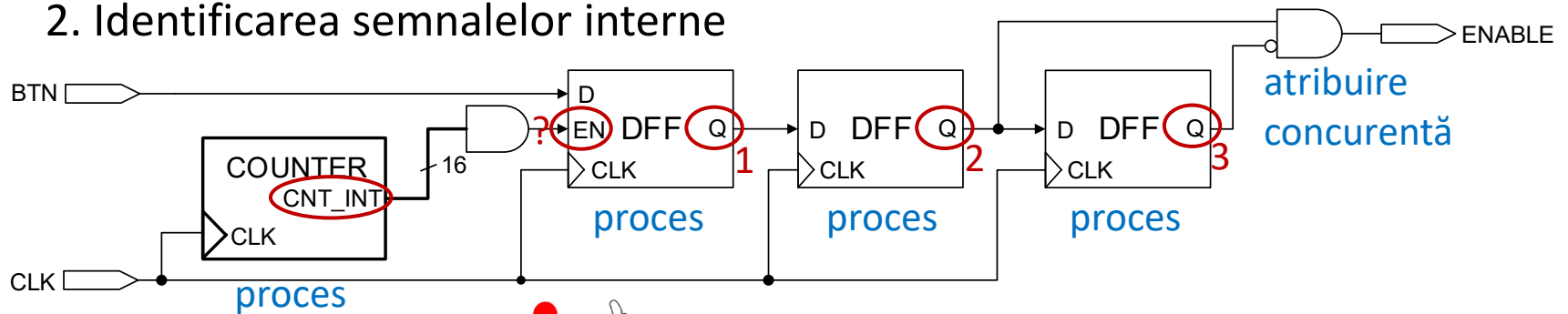
architecture ...

Descrieți totul într-o entitate MPG, fără alte entități suplimentare!

1.6 Descrierea în VHDL (recapitulare)

1.6.4 De la schemă la cod VHDL – Generatorul de monoimpuls (cont)

2. Identificarea semnalelor interne



entity MPG is
...
end MPG;



Intrările D la registre se pot înlocui cu sursele lor. Astfel, eliminăm redundanța => cod mai simplu.

architecture Behavioral of MPG is

signal cnt_int : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');

signal Q1, Q2, Q3 : STD_LOGIC;

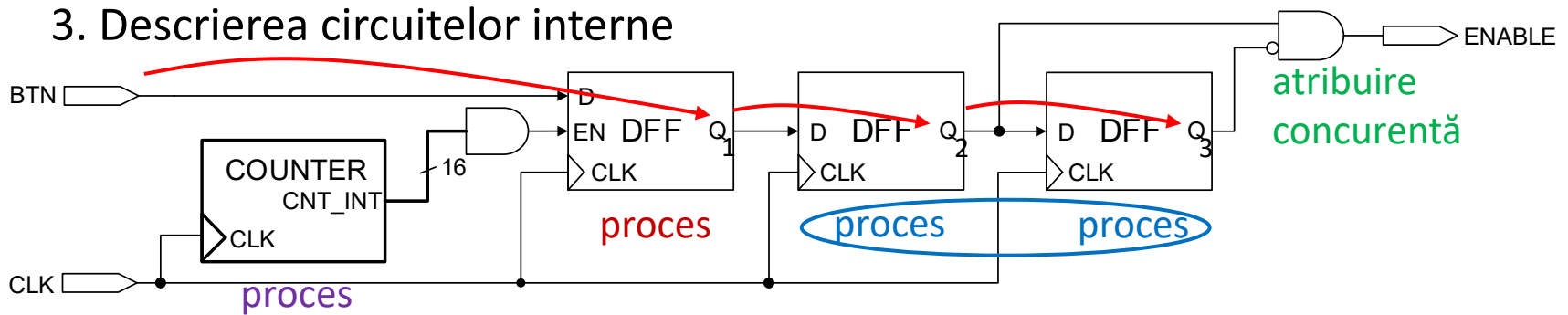
begin

-- ... descrierea comportamentului cu procese și atribuiri concurente

end Behavioral;

Descrieți totul într-o entitate MPG, fără alte entități suplimentare!

3. Descrierea circuitelor interne



Poarta ȘI de la final $\Rightarrow$ **begin**
`enable <= Q2 and (not Q3);`
`process(clk)`

Numărătorul  if clk_event and clk= 1 then
cnt_int <= cnt_int + 1;
end if;
end process;

Registrul 1
validat de
poarta ŞI
inclusă

begin
if clk'event and clk='1' then
if cnt_int(15 downto 0) = "1111111111111111" then
Q1 <= btn;
end if;
end if;
end process;

Poarta ŞI
un comp

Poarta $\mathcal{I}l$ este descrisă ca un comparator echivalent.

Registrele 2 și 3
comasate într-un
proces

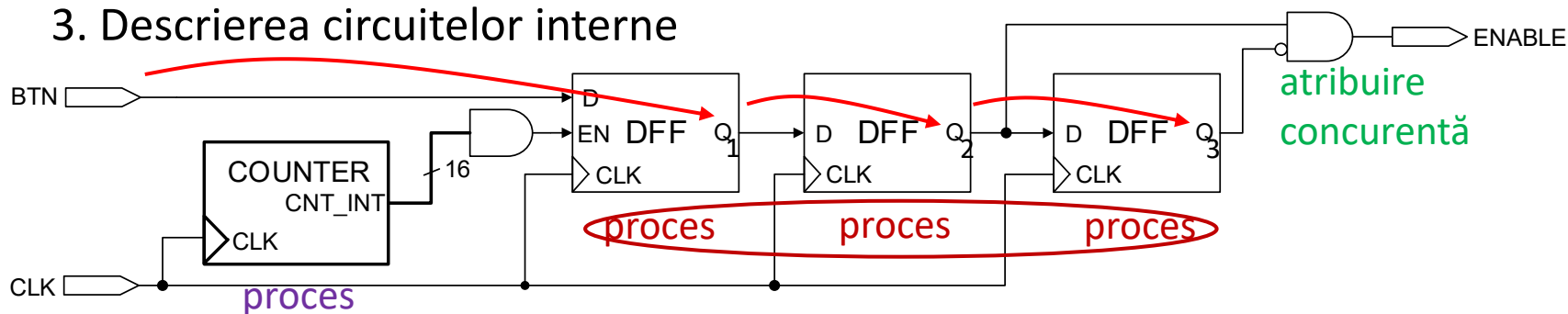
→

```
begin
    if clk'event and clk='1' then
        Q2 <= Q1;
        Q3 <= Q2;
    end if;
end process;
```

Î: Q3 <= Q1 ?
R: **NU!!!**

Regulă: Într-un proces semnalele primesc **ultima** valoare atribuită, numai după **suspendarea** lui.

3. Descrierea circuitelor interne



Variantă echivalentă cu cele 3 registre comasate într-un proces

```

begin
  Poarta ȘI de la final ➡ enable <= Q2 and (not Q3);
  process(clk)
  begin
    if clk'event and clk='1' then
      Numărătorul ➡ cnt_int <= cnt_int + 1;
    end if;
  end process;
  process(clk)
  begin
    if clk'event and clk='1' then
      Registrele 1, 2 ➡ if cnt_int(15 downto 0) = "1111111111111111" then
        și 3 comasate ➡ Q1 <= btn;
        într-un proces ➡ Q2 <= Q1;
        ➡ Q3 <= Q2;
      end if;
    end process;
  end Behavioral;
  
```

Referințe

- [1] F. Oniga, “De la bit la procesor. Introducere în arhitectura calculatoarelor”, Cap. 3.1, 5.1, 5.3, 6.1, 6.2, Ed. U.T. Press, 2019, ISBN 978-606-737-366-0. (pt. recapitulare)
- [2] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 1.7-1.8, 1.10-1.11, Anexa B2-B3, Ed. Morgan–Kaufmann, 2013.
- [3] J. L. Hennessy, D. A. Patterson, “Computer Organization and Design: A Quantitative Approach”, 5th edition, Cap. 1.1-1.3, 1.9, Anexa A1-A4, Ed. Morgan–Kaufmann, 2011.